

二维计算几何模板

原文：[计算几何基本模板（二维）](#)

原作者：Oneway`

原文发布时间：2023-08-19

本文在原文知识点基础上重新整理，统一为 C++17 接口，并补充边界条件与使用说明。

目录

- [使用约定](#)
- [统一模板](#)
- [基本设置](#)
- [点与向量](#)
- [直线与线段](#)
- [多边形](#)
- [圆](#)
- [网格与 Pick 定理](#)
- [极角排序](#)
- [凸包 Andrew 算法](#)
- [最小圆覆盖](#)
- [圆的面积并](#)
- [圆与多边形的面积交](#)
- [自适应 Simpson 积分](#)
- [平面最近点对](#)
- [常见问题与修正](#)
- [参考资料](#)

使用约定

- 默认使用 `double`，误差常量为 $EPS = 1e-9$ ，可按题目精度调整。
- 所有角度均使用弧度。
- 逆时针方向为正方向。
- 多边形顶点默认按顺时针或逆时针连续给出，不重复首点。
- 浮点数不能直接依赖 `==` 判断几何相等。
- 排序比较器必须满足严格弱序，因此字典序排序不使用 `EPS`。

- 对整数坐标的叉积、面积和 Pick 定理，建议使用 `long long` 或 `__int128`。
- `long double` 的输入输出格式为 `%Lf`，对应函数包括 `fabsl`、`sqrtl`、`cosl`。

统一模板

```
#include <bits/stdc++.h>
using namespace std;

namespace geometry {

using db = double;

constexpr db EPS = 1e-9;
const db PI = acos(-1.0);
const db INF = numeric_limits<db>::infinity();

int sgn(db x) {
    if (x < -EPS) return -1;
    if (x > EPS) return 1;
    return 0;
}

db clampUnit(db x) {
    return max(-1.0, min(1.0, x));
}

struct Point {
    db x = 0, y = 0;

    Point() = default;
    Point(db x, db y) : x(x), y(y) {}

    Point operator+ (const Point& p) const { return {x + p.x, y + p.y}; }
    Point operator- (const Point& p) const { return {x - p.x, y - p.y}; }
    Point operator* (db k) const { return {x * k, y * k}; }
    Point operator/ (db k) const { return {x / k, y / k}; }

    Point& operator+= (const Point& p) {
        x += p.x;
        y += p.y;
        return *this;
    }

    Point& operator-= (const Point& p) {
        x -= p.x;
```

```

        y -= p.y;
        return *this;
    }

    // 仅用于 sort: 不能在比较器中使用 EPS。
    bool operator< (const Point& p) const {
        return x != p.x ? x < p.x : y < p.y;
    }

    bool operator== (const Point& p) const {
        return sgn(x - p.x) == 0 && sgn(y - p.y) == 0;
    }

    bool operator!= (const Point& p) const {
        return !(*this == p);
    }
};

using Vector = Point;

Point operator* (db k, const Point& p) {
    return p * k;
}

// 点积:  $a \cdot b = |a||b|\cos(\theta)$ 。
db dot(Vector a, Vector b) {
    return a.x * b.x + a.y * b.y;
}

// 叉积:  $a \times b = |a||b|\sin(\theta)$ 。
db cross(Vector a, Vector b) {
    return a.x * b.y - a.y * b.x;
}

// 有向面积的两倍:  $(b-a) \times (c-a)$ 。
db cross(Point a, Point b, Point c) {
    return cross(b - a, c - a);
}

db length2(Vector a) {
    return dot(a, a);
}

```

```

db length(Vector a) {
    return sqrt(length2(a));
}

db distance(Point a, Point b) {
    return length(a - b);
}

// 零向量没有单位向量，调用前应保证 length(a) > EPS。
Vector normalize(Vector a) {
    return a / length(a);
}

// 返回两个非零向量的夹角，范围 [0, PI]。
db angle(Vector a, Vector b) {
    return acos(clampUnit(dot(a, b) / length(a) / length(b)));
}

// 返回从 a 逆时针转到 b 的有向角，范围 [-PI, PI]。
db directedAngle(Vector a, Vector b) {
    return atan2(cross(a, b), dot(a, b));
}

// 将向量逆时针旋转 rad。
Vector rotate(Vector a, db rad) {
    db c = cos(rad), s = sin(rad);
    return {a.x * c - a.y * s, a.x * s + a.y * c};
}

// 点 c 在有向直线 a->b 左侧返回 1，右侧返回 -1，共线返回 0。
int orientation(Point a, Point b, Point c) {
    return sgn(cross(a, b, c));
}

struct Line {
    Point p;
    Vector v;

    Line() = default;

    // 由直线上一点和方向向量构造。
    Line(Point p, Vector v) : p(p), v(v) {}
}

```

```

// 由直线上两点构造。
static Line through(Point a, Point b) {
    return {a, b - a};
}

Point point(db t) const {
    return p + v * t;
}
};

struct Segment {
    Point a, b;
};

// 三点共线。
bool collinear(Point a, Point b, Point c) {
    return orientation(a, b, c) == 0;
}

// 点到直线的距离。要求直线方向向量非零。
db distancePointLine(Point p, const Line& l) {
    return fabs(cross(l.v, p - l.p)) / length(l.v);
}

// 点到线段的距离，允许退化线段。
db distancePointSegment(Point p, Point a, Point b) {
    if (a == b) return distance(p, a);

    Vector ab = b - a;
    if (sgn(dot(ab, p - a)) < 0) return distance(p, a);
    if (sgn(dot(ab, p - b)) > 0) return distance(p, b);
    return fabs(cross(ab, p - a)) / length(ab);
}

// includeEndpoints=true 时包含端点。
bool onSegment(Point p, Point a, Point b, bool includeEndpoints = true) {
    if (orientation(a, b, p) != 0) return false;
    int d = sgn(dot(a - p, b - p));
    return includeEndpoints ? d <= 0 : d < 0;
}

// 无限直线 ab 是否与线段 cd 有公共点。
bool lineIntersectsSegment(Point a, Point b, Point c, Point d) {

```

```

    int x = orientation(a, b, c);
    int y = orientation(a, b, d);
    return x == 0 || y == 0 || x != y;
}

```

// 两闭线段是否相交，包括端点接触与共线重叠。

```

bool segmentsIntersect(Point a, Point b, Point c, Point d) {
    int abC = orientation(a, b, c);
    int abD = orientation(a, b, d);
    int cdA = orientation(c, d, a);
    int cdB = orientation(c, d, b);

    if (abC == 0 && onSegment(c, a, b)) return true;
    if (abD == 0 && onSegment(d, a, b)) return true;
    if (cdA == 0 && onSegment(a, c, d)) return true;
    if (cdB == 0 && onSegment(b, c, d)) return true;

    return abC * abD < 0 && cdA * cdB < 0;
}

```

```

bool parallel(const Line& a, const Line& b) {
    return sgn(cross(a.v, b.v)) == 0;
}

```

// 求两条不平行直线的唯一交点。

// 平行或重合时返回 false。

```

bool lineIntersection(const Line& a, const Line& b, Point& out) {
    db det = cross(a.v, b.v);
    if (sgn(det) == 0) return false;

    db t = cross(b.p - a.p, b.v) / det;
    out = a.point(t);
    return true;
}

```

```

db triangleSignedArea(Point a, Point b, Point c) {
    return cross(a, b, c) / 2.0;
}

```

```

db triangleArea(Point a, Point b, Point c) {
    return fabs(triangleSignedArea(a, b, c));
}

```

// 海伦公式。叉积公式通常更快、更稳定。

```
db triangleAreaHeron(Point a, Point b, Point c) {  
    db x = distance(a, b);  
    db y = distance(a, c);  
    db z = distance(b, c);  
    db p = (x + y + z) / 2;  
    return sqrt(max(0.0, p * (p - x) * (p - y) * (p - z)));  
}
```

// 多边形有向面积：逆时针为正，顺时针为负。

```
db polygonSignedArea(const vector<Point>& polygon) {  
    db area2 = 0;  
    int n = static_cast<int>(polygon.size());  
  
    for (int i = 0; i < n; ++i) {  
        area2 += cross(polygon[i], polygon[(i + 1) % n]);  
    }  
    return area2 / 2;  
}
```

```
db polygonArea(const vector<Point>& polygon) {  
    return fabs(polygonSignedArea(polygon));  
}
```

```
enum class PointPolygonRelation {  
    Outside,  
    Boundary,  
    Inside  
};
```

// 射线法，适用于简单多边形，包括凹多边形。

// 复杂度 $O(n)$ ，顶点顺序可顺时针或逆时针。

```
PointPolygonRelation pointInPolygon(  
    Point p,  
    const vector<Point>& polygon  
) {  
    bool inside = false;  
    int n = static_cast<int>(polygon.size());  
  
    for (int i = 0, j = n - 1; i < n; j = i++) {  
        Point a = polygon[j];  
        Point b = polygon[i];
```



```

    if (onSegment(p, a, b)) {
        return PointPolygonRelation::Boundary;
    }

    bool crossesY = (a.y > p.y) != (b.y > p.y);
    if (!crossesY) continue;

    db intersectionX =
        a.x + (b.x - a.x) * (p.y - a.y) / (b.y - a.y);

    if (intersectionX > p.x) inside = !inside;
}

return inside
    ? PointPolygonRelation::Inside
    : PointPolygonRelation::Outside;
}

// 判断按边界顺序给出的多边形是否为凸多边形。
// strict=false 时允许连续三点共线。
bool isConvex(const vector<Point>& polygon, bool strict = false) {
    int n = static_cast<int>(polygon.size());
    if (n < 3) return false;

    int direction = 0;
    for (int i = 0; i < n; ++i) {
        int turn = orientation(
            polygon[i],
            polygon[(i + 1) % n],
            polygon[(i + 2) % n]
        );

        if (turn == 0) {
            if (strict) return false;
            continue;
        }

        if (direction == 0) direction = turn;
        else if (direction != turn) return false;
    }

    return direction != 0;
}

```

```

struct Circle {
    Point center;
    db radius = 0;

    Circle() = default;
    Circle(Point center, db radius) : center(center), radius(radius) {}

    db area() const {
        return PI * radius * radius;
    }

    db circumference() const {
        return 2 * PI * radius;
    }
};

```

```

// 弧长  $L = \alpha * r$ , 扇形面积  $S = \alpha * r^2 / 2$ 。
db sectorArea(Vector a, Vector b, db radius) {
    return radius * radius * directedAngle(a, b) / 2;
}

```

```

enum class PointCircleRelation {
    Outside = -1,
    On = 0,
    Inside = 1
};

```

```

PointCircleRelation relation(Point p, const Circle& c) {
    int state = sgn(distance(p, c.center) - c.radius);
    if (state > 0) return PointCircleRelation::Outside;
    if (state < 0) return PointCircleRelation::Inside;
    return PointCircleRelation::On;
}

```

```

enum class LineCircleRelation {
    Separate = -1,
    Tangent = 0,
    Intersect = 1
};

```

```

LineCircleRelation relation(const Line& line, const Circle& c) {
    int state = sgn(distancePointLine(c.center, line) - c.radius);
}

```

```

    if (state > 0) return LineCircleRelation::Separate;
    if (state < 0) return LineCircleRelation::Intersect;
    return LineCircleRelation::Tangent;
}

```

// 返回直线与圆的交点，数量为 0、1 或 2。

```

vector<Point> lineCircleIntersections(const Line& line, const Circle& c) {
    db vv = length2(line.v);
    if (sgn(vv) == 0) return {};

    db t = dot(c.center - line.p, line.v) / vv;
    Point projection = line.point(t);
    db h2 = c.radius * c.radius - length2(projection - c.center);

    if (sgn(h2) < 0) return {};
    if (sgn(h2) == 0) return {projection};

    Vector offset = normalize(line.v) * sqrt(max(0.0, h2));
    return {projection - offset, projection + offset};
}

```

```

enum class CircleCircleRelation {
    Separate,
    ExternallyTangent,
    Intersect,
    InternallyTangent,
    Contained,
    Coincident
};

```

```

CircleCircleRelation relation(const Circle& a, const Circle& b) {
    db d = distance(a.center, b.center);

    if (sgn(d) == 0 && sgn(a.radius - b.radius) == 0) {
        return CircleCircleRelation::Coincident;
    }

    if (sgn(d - (a.radius + b.radius)) > 0) {
        return CircleCircleRelation::Separate;
    }

    if (sgn(d - (a.radius + b.radius)) == 0) {
        return CircleCircleRelation::ExternallyTangent;
    }
}

```

```

}

db radiusDifference = fabs(a.radius - b.radius);

if (sgn(d - radiusDifference) < 0) {
    return CircleCircleRelation::Contained;
}

if (sgn(d - radiusDifference) == 0) {
    return CircleCircleRelation::InternallyTangent;
}

return CircleCircleRelation::Intersect;
}

// 返回两圆的有限交点。
// 重合圆有无穷多个交点，此时返回空数组并应单独判断 relation。
vector<Point> circleCircleIntersections(const Circle& a, const Circle& b) {
    db d = distance(a.center, b.center);

    if (sgn(d) == 0) return {};
    if (sgn(d - a.radius - b.radius) > 0) return {};
    if (sgn(d - fabs(a.radius - b.radius)) < 0) return {};

    db x =
        (a.radius * a.radius - b.radius * b.radius + d * d) /
        (2 * d);

    db h2 = a.radius * a.radius - x * x;
    Vector direction = (b.center - a.center) / d;
    Point base = a.center + direction * x;

    if (sgn(h2) == 0) return {base};

    Vector normal = rotate(direction, PI / 2) * sqrt(max(0.0, h2));
    return {base - normal, base + normal};
}

// 返回点到圆的切点，数量为 0、1 或 2。
vector<Point> tangentPoints(Point p, const Circle& c) {
    Vector v = p - c.center;
    db d2 = length2(v);
    db r2 = c.radius * c.radius;

```

```

    if (sgn(d2 - r2) < 0) return {};
    if (sgn(d2 - r2) == 0) return {p};

    Point base = c.center + v * (r2 / d2);
    Vector offset =
        rotate(v, PI / 2) *
        (c.radius * sqrt(max(0.0, d2 - r2)) / d2);

    return {base - offset, base + offset};
}

// 三点外接圆。三点共线时返回 false。
bool circumcircle(Point a, Point b, Point c, Circle& out) {
    Vector u = b - a;
    Vector v = c - a;
    db determinant = 2 * cross(u, v);

    if (sgn(determinant) == 0) return false;

    db uu = length2(u);
    db vv = length2(v);

    Point center{
        a.x + (v.y * uu - u.y * vv) / determinant,
        a.y + (u.x * vv - v.x * uu) / determinant
    };

    out = Circle(center, distance(center, a));
    return true;
}

// 三角形内切圆。三点共线时返回 false。
bool incircle(Point a, Point b, Point c, Circle& out) {
    db sideA = distance(b, c);
    db sideB = distance(a, c);
    db sideC = distance(a, b);
    db perimeter = sideA + sideB + sideC;

    if (sgn(perimeter) == 0 || orientation(a, b, c) == 0) {
        return false;
    }
}

```

```

Point center =
    (a * sideA + b * sideB + c * sideC) / perimeter;

out = Circle(
    center,
    distancePointLine(center, Line::through(a, b))
);
return true;
}

```

// 极角排序：以 origin 为极点，从 x 轴正方向开始逆时针排序。
 // 同方向的点按到极点距离从小到大排列。

```

void polarSort(vector<Point>& points, Point origin = {0, 0}) {
    auto half = [](Vector v) {
        return v.y > 0 || (v.y == 0 && v.x >= 0) ? 0 : 1;
    };

    sort(points.begin(), points.end(), [&](Point a, Point b) {
        Vector u = a - origin;
        Vector v = b - origin;

        int hu = half(u);
        int hv = half(v);
        if (hu != hv) return hu < hv;

        db product = cross(u, v);
        if (product != 0) return product > 0;
        return length2(u) < length2(v);
    });
}

```

// Andrew 凸包，复杂度 $O(n \log n)$ 。

// keepCollinear=false 时删除边上的中间共线点。

```

vector<Point> convexHull(
    vector<Point> points,
    bool keepCollinear = false
) {
    sort(points.begin(), points.end());
    points.erase(unique(points.begin(), points.end()), points.end());

    int n = static_cast<int>(points.size());
    if (n <= 1) return points;
}

```

```

vector<Point> hull;

auto shouldPop = [&](Point a, Point b, Point c) {
    int turn = orientation(a, b, c);
    return keepCollinear ? turn < 0 : turn <= 0;
};

for (Point p : points) {
    while (hull.size() >= 2 &&
        shouldPop(hull[hull.size() - 2], hull.back(), p)) {
        hull.pop_back();
    }
    hull.push_back(p);
}

size_t lowerSize = hull.size();

for (int i = n - 2; i >= 0; --i) {
    Point p = points[i];
    while (hull.size() > lowerSize &&
        shouldPop(hull[hull.size() - 2], hull.back(), p)) {
        hull.pop_back();
    }
    hull.push_back(p);
}

hull.pop_back();
return hull;
}

bool contains(const Circle& c, Point p) {
    return sgn(distance(c.center, p) - c.radius) <= 0;
}

Circle diameterCircle(Point a, Point b) {
    Point center = (a + b) / 2;
    return {center, distance(a, b) / 2};
}

// 三个边界点确定的最小覆盖圆。
// 共线时取三对直径圆中能够覆盖三点且半径最小者。
Circle minimumCircleForThree(Point a, Point b, Point c) {
    Circle result;

```

```

    if (circumcircle(a, b, c, result)) return result;

    vector<Circle> candidates = {
        diameterCircle(a, b),
        diameterCircle(a, c),
        diameterCircle(b, c)
    };

    Circle best{{0, 0}, INF};
    for (const Circle& candidate : candidates) {
        if (contains(candidate, a) &&
            contains(candidate, b) &&
            contains(candidate, c) &&
            candidate.radius < best.radius) {
            best = candidate;
        }
    }
    return best;
}

// 随机增量最小圆覆盖，期望复杂度  $O(n)$ 。
Circle minimumEnclosingCircle(vector<Point> points) {
    if (points.empty()) return {{0, 0}, 0};

    mt19937 rng(
        static_cast<unsigned>(
            chrono::steady_clock::now().time_since_epoch().count()
        )
    );
    shuffle(points.begin(), points.end(), rng);

    Circle result{points[0], 0};

    for (int i = 0; i < static_cast<int>(points.size()); ++i) {
        if (contains(result, points[i])) continue;

        result = {points[i], 0};

        for (int j = 0; j < i; ++j) {
            if (contains(result, points[j])) continue;

            result = diameterCircle(points[i], points[j]);
        }
    }
}

```



```

        for (int k = 0; k < j; ++k) {
            if (contains(result, points[k])) continue;

            result = minimumCircleForThree(
                points[i],
                points[j],
                points[k]
            );
        }
    }

    return result;
}

// 圆面积并，复杂度  $O(n^2 \log n)$ 。
// 使用每个圆未被其他圆覆盖的边界弧及格林公式求面积。
db circleUnionArea(vector<Circle> circles) {
    vector<Circle> filtered;

    for (int i = 0; i < static_cast<int>(circles.size()); ++i) {
        bool covered = false;

        for (int j = 0; j < static_cast<int>(circles.size()); ++j) {
            if (i == j) continue;

            db d = distance(circles[i].center, circles[j].center);

            if (sgn(
                d + circles[i].radius - circles[j].radius
            ) <= 0 &&
                (sgn(circles[i].radius - circles[j].radius) < 0 ||
                 i > j)) {
                covered = true;
                break;
            }
        }

        if (!covered && sgn(circles[i].radius) > 0) {
            filtered.push_back(circles[i]);
        }
    }
}

```

```

db twiceArea = 0;

for (int i = 0; i < static_cast<int>(filtered.size()); ++i) {
    const Circle& current = filtered[i];
    vector<pair<db, int>> events;
    int active = 0;

    events.push_back({-PI, 0});
    events.push_back({PI, 0});

    for (int j = 0; j < static_cast<int>(filtered.size()); ++j) {
        if (i == j) continue;

        const Circle& other = filtered[j];
        db d = distance(current.center, other.center);

        if (sgn(d - current.radius - other.radius) >= 0) continue;
        if (sgn(d + other.radius - current.radius) <= 0) continue;

        db centerAngle = atan2(
            other.center.y - current.center.y,
            other.center.x - current.center.x
        );

        db delta = acos(clampUnit(
            (current.radius * current.radius + d * d -
             other.radius * other.radius) /
            (2 * current.radius * d)
        ));

        db left = centerAngle - delta;
        db right = centerAngle + delta;

        while (left < -PI) {
            left += 2 * PI;
            right += 2 * PI;
        }
        while (left >= PI) {
            left -= 2 * PI;
            right -= 2 * PI;
        }

        if (right <= PI) {

```

```

        events.push_back({left, 1});
        events.push_back({right, -1});
    } else {
        ++active;
        events.push_back({left, 1});
        events.push_back({PI, -1});
        events.push_back({-PI, 1});
        events.push_back({right - 2 * PI, -1});
    }
}

sort(events.begin(), events.end());

for (int k = 0; k + 1 < static_cast<int>(events.size()); ++k) {
    active += events[k].second;
    db left = events[k].first;
    db right = events[k + 1].first;

    if (active != 0 || right <= left) continue;

    Point o = current.center;
    db r = current.radius;

    twiceArea += r * (
        r * (right - left) +
        o.x * (sin(right) - sin(left)) -
        o.y * (cos(right) - cos(left))
    );
}

return fabs(twiceArea) / 2;
}

```

// 原点圆与有向线段 ab 对面积交的贡献。

```

db circleTriangleIntersection(Vector a, Vector b, db radius) {
    Vector d = b - a;
    vector<db> cuts = {0, 1};

    db A = dot(d, d);
    db B = 2 * dot(a, d);
    db C = dot(a, a) - radius * radius;
    db discriminant = B * B - 4 * A * C;
}

```

```

    if (sgn(A) != 0 && sgn(discriminant) >= 0) {
        db root = sqrt(max(0.0, discriminant));
        db t1 = (-B - root) / (2 * A);
        db t2 = (-B + root) / (2 * A);

        if (sgn(t1) > 0 && sgn(t1 - 1) < 0) cuts.push_back(t1);
        if (sgn(t2) > 0 && sgn(t2 - 1) < 0) cuts.push_back(t2);
    }

    sort(cuts.begin(), cuts.end());

    db area = 0;

    for (int i = 0; i + 1 < static_cast<int>(cuts.size()); ++i) {
        db left = cuts[i];
        db right = cuts[i + 1];

        Vector u = a + d * left;
        Vector v = a + d * right;
        Vector middle = a + d * ((left + right) / 2);

        if (sgn(length(middle) - radius) <= 0) {
            area += cross(u, v) / 2;
        } else {
            area += radius * radius *
                atan2(cross(u, v), dot(u, v)) / 2;
        }
    }

    return area;
}

// 圆与简单多边形的面积交，复杂度 O(n)。
db circlePolygonIntersectionArea(
    const Circle& circle,
    const vector<Point>& polygon
) {
    db area = 0;
    int n = static_cast<int>(polygon.size());

    for (int i = 0; i < n; ++i) {
        Vector a = polygon[i] - circle.center;

```

```

        Vector b = polygon[(i + 1) % n] - circle.center;
        area += circleTriangleIntersection(a, b, circle.radius);
    }

    return fabs(area);
}

```

// 通用自适应 Simpson 积分。

```

template <class Function>
class AdaptiveSimpson {
private:
    Function function;
    db tolerance;
    int maxDepth;

    db simpson(db left, db right, db fLeft, db fMid, db fRight) {
        return (right - left) *
            (fLeft + 4 * fMid + fRight) / 6;
    }

    db solve(
        db left,
        db right,
        db fLeft,
        db fMid,
        db fRight,
        db whole,
        db eps,
        int depth
    ) {
        db middle = (left + right) / 2;
        db leftMiddle = (left + middle) / 2;
        db rightMiddle = (middle + right) / 2;

        db fLeftMiddle = function(leftMiddle);
        db fRightMiddle = function(rightMiddle);

        db leftArea = simpson(
            left,
            middle,
            fLeft,
            fLeftMiddle,
            fMid

```

```

);

db rightArea = simpson(
    middle,
    right,
    fMid,
    fRightMiddle,
    fRight
);

db delta = leftArea + rightArea - whole;

if (depth <= 0 || fabs(delta) <= 15 * eps) {
    return leftArea + rightArea + delta / 15;
}

return solve(
    left,
    middle,
    fLeft,
    fLeftMiddle,
    fMid,
    leftArea,
    eps / 2,
    depth - 1
) +
solve(
    middle,
    right,
    fMid,
    fRightMiddle,
    fRight,
    rightArea,
    eps / 2,
    depth - 1
);
}

```

public:

```

AdaptiveSimpson(
    Function function,
    db tolerance = 1e-9,
    int maxDepth = 25

```

```

)
: function(function),
  tolerance(tolerance),
  maxDepth(maxDepth) {}

db integrate(db left, db right) {
  if (left == right) return 0;
  if (left > right) return -integrate(right, left);

  db middle = (left + right) / 2;
  db fLeft = function(left);
  db fMid = function(middle);
  db fRight = function(right);

  db whole = simpson(
    left,
    right,
    fLeft,
    fMid,
    fRight
  );

  return solve(
    left,
    right,
    fLeft,
    fMid,
    fRight,
    whole,
    tolerance,
    maxDepth
  );
}
};

```

// 平面最近点对，复杂度 $O(n \log n)$ 。

```

db closestPairDistance(vector<Point> points) {
  int n = static_cast<int>(points.size());
  if (n < 2) return INF;

  sort(points.begin(), points.end());
  vector<Point> buffer(n);

```

```

function<db(int, int)> solve = [&](int left, int right) -> db {
    if (right - left <= 3) {
        db answer = INF;

        for (int i = left; i < right; ++i) {
            for (int j = i + 1; j < right; ++j) {
                answer = min(answer, distance(points[i], points[j]));
            }
        }

        sort(
            points.begin() + left,
            points.begin() + right,
            [](Point a, Point b) {
                return a.y != b.y ? a.y < b.y : a.x < b.x;
            }
        );

        return answer;
    }

    int middle = (left + right) / 2;
    db middleX = points[middle].x;

    db answer = min(
        solve(left, middle),
        solve(middle, right)
    );

    merge(
        points.begin() + left,
        points.begin() + middle,
        points.begin() + middle,
        points.begin() + right,
        buffer.begin(),
        [](Point a, Point b) {
            return a.y != b.y ? a.y < b.y : a.x < b.x;
        }
    );

    copy(
        buffer.begin(),
        buffer.begin() + (right - left),

```



```

        points.begin() + left
    );

    vector<Point> strip;
    strip.reserve(right - left);

    for (int i = left; i < right; ++i) {
        if (fabs(points[i].x - middleX) < answer) {
            strip.push_back(points[i]);
        }
    }

    for (int i = 0; i < static_cast<int>(strip.size()); ++i) {
        for (int j = i + 1;
            j < static_cast<int>(strip.size()) &&
            strip[j].y - strip[i].y < answer;
            ++j) {
            answer = min(answer, distance(strip[i], strip[j]));
        }
    }

    return answer;
};

return solve(0, n);
}

} // namespace geometry

```

基本设置

浮点误差

计算几何中通常不能直接使用 $x == 0$ 。统一使用符号函数：

```

int sgn(double x) {
    if (x < -EPS) return -1;
    if (x > EPS) return 1;
    return 0;
}

```

EPS 应根据坐标范围和题目精度调整。坐标很大且运算层数较多时，可以使用 long double。

long double 输入输出

```
long double x;  
scanf("%Lf", &x);  
printf("%.10Lf\n", x);
```

```
// 常用函数  
fabsl(x);  
sqrtl(x);  
cosl(x);  
sinl(x);
```

点与向量

点 $A(x_1, y_1)$ 、 $B(x_2, y_2)$ 对应向量：

$$\overrightarrow{AB} = B - A = (x_2 - x_1, y_2 - y_1)$$

点积

$$\vec{a} \cdot \vec{b} = |\vec{a}| |\vec{b}| \cos \theta = x_1 x_2 + y_1 y_2$$

点积可以判断夹角：

- $\theta = 0^\circ$ ：点积为 $|\vec{a}| |\vec{b}|$ 。
- $\theta = 180^\circ$ ：点积为 $-|\vec{a}| |\vec{b}|$ 。
- $\theta < 90^\circ$ ：点积大于 0。
- $\theta = 90^\circ$ ：点积等于 0。
- $\theta > 90^\circ$ ：点积小于 0。

叉积

$$\vec{a} \times \vec{b} = |\vec{a}| |\vec{b}| \sin \theta = x_1 y_2 - x_2 y_1$$

cross(b-a, c-a)：

- 大于 0：c 在有向直线 a -> b 左侧。
- 小于 0：c 在右侧。

- 等于 0：三点共线。

距离、模长和单位向量

$$AB = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

$$|\vec{a}| = \sqrt{x^2 + y^2}$$

$$\text{norm}(\vec{a}) = \frac{\vec{a}}{|\vec{a}|}$$

零向量没有单位向量，调用 `normalize` 前必须保证模长非零。

向量夹角

$$\theta = \arccos \frac{\vec{a} \cdot \vec{b}}{|\vec{a}| |\vec{b}|}$$

由于浮点误差，传入 `acos` 的参数应限制到 `[-1,1]`。

向量旋转

向量 (x, y) 逆时针旋转 θ ：

$$(x \cos \theta - y \sin \theta, x \sin \theta + y \cos \theta)$$

直线与线段

常见直线表达式：

- 一般式： $ax + by + c = 0$ 。
- 点向式：一点 (x_0, y_0) 和方向向量 (u, v) 。
- 斜截式： $y = kx + b$ 。

模板使用点向式：

```
Line line(point, direction);  
Line line = Line::through(a, b);
```

三点共线

三点 A, B, C 共线，当且仅当：

$$\overrightarrow{AB} \times \overrightarrow{AC} = 0$$

点到直线距离

$$d = \frac{|\overrightarrow{AB} \times \overrightarrow{AP}|}{|\overrightarrow{AB}|}$$

点到线段距离

- 投影在线段内部：取点到所在直线的垂直距离。
- 投影在线段外部：取到两个端点距离的较小值。
- 线段退化成点：直接计算两点距离。

线段相交

闭线段相交需要同时处理：

- 规范相交。
- 端点接触。
- 共线且区间重叠。
- 退化线段。

直线交点

设两直线分别为：

$$P + t\vec{u}, \quad Q + s\vec{v}$$

当 $\vec{u} \times \vec{v} \neq 0$ 时存在唯一交点。平行或重合时不能直接除以叉积。

多边形

三角形面积

海伦公式：

$$S = \sqrt{p(p-a)(p-b)(p-c)}, \quad p = \frac{a+b+c}{2}$$

叉积公式：

$$S_{\triangle ABC} = \frac{1}{2} \left| \overrightarrow{AB} \times \overrightarrow{AC} \right|$$

已知两边及夹角：

$$S = \frac{1}{2} ab \sin C$$

三角形四心

- 外心：三边中垂线交点，也是外接圆圆心。
- 内心：三条角平分线交点，也是内切圆圆心。
- 垂心：三条高的交点。
- 重心：三条中线交点。

重心坐标：

$$G = \left(\frac{x_1 + x_2 + x_3}{3}, \frac{y_1 + y_2 + y_3}{3} \right)$$

正弦定理

$$\frac{a}{\sin A} = \frac{b}{\sin B} = \frac{c}{\sin C} = 2R$$

其中 R 为外接圆半径。

余弦定理

$$a^2 = b^2 + c^2 - 2bc \cos A$$

$$b^2 = a^2 + c^2 - 2ac \cos B$$

$$c^2 = a^2 + b^2 - 2ab \cos C$$

正多边形

正 n 边形：

- 内角和： $(n-2) \times 180^\circ$ 。

- 每个内角： $\frac{(n-2)180^\circ}{n}$ 。
- 外角和： 360° 。
- 每个外角： $\frac{360^\circ}{n}$ 。
- 每个中心角： $\frac{360^\circ}{n}$ 。
- 对角线数量： $\frac{n(n-3)}{2}$ 。

正多边形的外接圆圆心、内切圆圆心和重心重合。

多边形面积与鞋带定理

$$S = \frac{1}{2} \left| \sum_{i=0}^{n-1} (x_i y_{i+1} - x_{i+1} y_i) \right|$$

其中下标对 n 取模。

有向面积：

- 逆时针顶点序列为正。
- 顺时针顶点序列为负。

点在多边形内

射线法适用于简单多边形，包括凹多边形：

- 射线与边界交点数为奇数：点在内部。
- 交点数为偶数：点在外部。
- 点在线段上时单独返回边界状态。
- 使用半开区间处理顶点，避免同一顶点被统计两次。

判断凸多边形

按边界顺序遍历所有连续三点：

- 所有非零转向符号相同，则多边形为凸多边形。
- 若允许共线边，忽略转向为零的位置。
- 若要求严格凸，则任何连续三点共线都返回 `false`。

圆

弧长公式：

$$L = \alpha r$$

圆面积与周长：

$$S = \pi r^2, \quad C = 2\pi r$$

扇形面积

$$S = \frac{1}{2}lr = \frac{1}{2}\alpha r^2$$

其中 α 必须使用弧度。

点与圆的位置关系

设点到圆心距离为 d ：

- $d < r$ ：圆内。
- $d = r$ ：圆上。
- $d > r$ ：圆外。

直线与圆的位置关系

设圆心到直线距离为 d ：

- $d < r$ ：相交。
- $d = r$ ：相切。
- $d > r$ ：相离。

直线与圆的交点

先求圆心到直线的垂足 H ，再计算：

$$h = \sqrt{r^2 - OH^2}$$

沿直线单位方向向量移动 $\pm h$ 即可得到交点。

圆与圆的位置关系

设圆心距为 d ，半径为 r_1, r_2 ：

- $d > r_1 + r_2$ ：相离。
- $d = r_1 + r_2$ ：外切。

- $|r_1 - r_2| < d < r_1 + r_2$: 相交。
- $d = |r_1 - r_2|$: 内切。
- $d < |r_1 - r_2|$: 内含。
- $d = 0$ 且 $r_1 = r_2$: 重合。

圆与圆的交点

两圆相交时，先在圆心连线上求公共弦中点，再沿垂直方向偏移。相切时只有一个交点；重合时有无穷多个交点。

圆外一点的切点

- 点在圆内：没有实切点。
- 点在圆上：切点就是该点。
- 点在圆外：存在两个切点。

三角形外接圆

外心到三个顶点距离相同。三点共线时不存在有限外接圆，模板通过 `bool` 返回值报告失败。

三角形内切圆

内心为按对边边长加权的顶点平均：

$$I = \frac{aA + bB + cC}{a + b + c}$$

内切圆半径为内心到任意一条边的距离。

网格与 Pick 定理

格点算法应使用整数坐标类型，不能将 `double` 坐标直接转换为整数。


```

namespace lattice {

using i64 = long long;
using i128 = __int128_t;

struct Point {
    i64 x, y;
};

// 线段 AB 上包含两个端点的格点数。
i64 pointsOnSegment(Point a, Point b) {
    i64 dx = labs(a.x - b.x);
    i64 dy = labs(a.y - b.y);
    return std::gcd(dx, dy) + 1;
}

// 简单格点多边形边界上的格点总数。
i64 boundaryPoints(const vector<Point>& polygon) {
    i64 result = 0;
    int n = static_cast<int>(polygon.size());

    for (int i = 0; i < n; ++i) {
        Point a = polygon[i];
        Point b = polygon[(i + 1) % n];
        result += std::gcd(
            labs(a.x - b.x),
            labs(a.y - b.y)
        );
    }

    return result;
}

// 返回面积的两倍。
i128 doubledArea(const vector<Point>& polygon) {
    i128 result = 0;
    int n = static_cast<int>(polygon.size());

    for (int i = 0; i < n; ++i) {
        Point a = polygon[i];
        Point b = polygon[(i + 1) % n];
        result += static_cast<i128>(a.x) * b.y
            - static_cast<i128>(a.y) * b.x;
    }
}

```

```

    }

    return result >= 0 ? result : -result;
}

// Pick 定理: 2A = 2I + B - 2。
i128 interiorPoints(const vector<Point>& polygon) {
    i128 area2 = doubledArea(polygon);
    i64 boundary = boundaryPoints(polygon);
    return (area2 - boundary + 2) / 2;
}

} // namespace lattice

```

线段两端点均为格点时：

- 包含端点的格点数： $\gcd(|\Delta x|, |\Delta y|) + 1$ 。
- 不包含端点的内部格点数： $\max(0, \gcd(|\Delta x|, |\Delta y|) - 1)$ 。

Pick 定理：

$$A = I + \frac{B}{2} - 1$$

其中：

- A ：多边形面积。
- I ：内部格点数。
- B ：边界格点数。

极角排序

极角排序通常需要确定：

1. 极点。
2. 起始射线。
3. 顺时针或逆时针。
4. 共线点按距离升序还是降序。

模板以极点 `origin` 为中心，从 x 轴正方向开始逆时针排序，共线同向点按距离升序排列。

比较器中不应使用带 `EPS` 的“近似相等”，否则可能破坏 `std::sort` 所要求的严格弱序。

凸包 Andrew 算法

给定平面点集，凸包是包含所有点的最小凸多边形。

Andrew 算法步骤：

1. 按 (x,y) 字典序排序。
2. 去除重复点。
3. 从左到右维护下凸包。
4. 从右到左维护上凸包。
5. 删除重复的首尾点。

复杂度：

$$O(n \log n)$$

`keepCollinear=false` 时只保留每条凸包边的两个端点；设置为 `true` 时保留边界上的共线点。

最小圆覆盖

问题：求半径最小的圆，使其覆盖给定的所有点。

随机增量算法的期望复杂度：

$$O(n)$$

增量过程中：

- 当前点在圆内：无需修改。
- 一个边界点：圆退化为该点。
- 两个边界点：以两点连线为直径。
- 三个不共线边界点：取外接圆。
- 三点共线：从三对直径圆中选择能覆盖三点的最小圆。

输入为空时，模板返回圆心 $(0,0)$ 、半径 0 。

圆的面积并

问题：求平面上若干圆覆盖区域的总面积。

模板使用格林公式，对每个圆统计未被其他圆覆盖的圆弧。复杂度为：

$$O(n^2 \log n)$$

主要步骤：

1. 删除被其他圆完全包含的圆。
2. 对每个圆计算被其他圆覆盖的角区间。
3. 合并角区间。
4. 对未覆盖圆弧应用格林公式。
5. 汇总所有边界弧贡献。

需要特别处理：

- 重复圆。
- 内含圆。
- 相切圆。
- 角区间跨越 $-\pi$ 与 π 。
- `acos` 参数的浮点越界。

圆与多边形的面积交

问题：求一个圆与简单多边形公共部分的面积。

将多边形沿圆心进行三角剖分。对每条有向边：

1. 把圆心平移到原点。
2. 求线段与圆的参数交点。
3. 将线段切分为若干小段。
4. 小段中点在圆内时累加三角形面积。
5. 小段中点在圆外时累加扇形面积。
6. 最后取所有有向贡献之和的绝对值。

复杂度：

$$O(n)$$

原文写作 $O(nT)$ ，其中每条边的交点数量 T 至多为常数，因此标准实现总体为 $O(n)$ 。

自适应 Simpson 积分

Simpson 公式：

$$\int_l^r f(x) dx \approx \frac{r-l}{6} \left[f(l) + 4f\left(\frac{l+r}{2}\right) + f(r) \right]$$

对二次多项式该公式精确成立。

自适应算法不断二分区间，比较：

$$S(l, m) + S(m, r)$$

与：

$$S(l, r)$$

的误差。常用 Richardson 修正为：

$$S_{\text{corrected}} = S_L + S_R + \frac{S_L + S_R - S}{15}$$

调用示例：

```
using namespace geometry;

auto function = [](double x) {
    return exp(-x * x);
};

AdaptiveSimpson solver(function, 1e-10);
double answer = solver.integrate(0, 1);
```

原文示例中的 `asr(1, r, 0)` 初值不正确。初始估计必须是整个区间的 Simpson 值，不能传入 `0`。

平面最近点对

给定平面上 n 个点，求任意两点之间的最小欧氏距离。

分治算法：

1. 初始按 x 排序。
2. 将区间分为左右两半。
3. 递归求左右答案。
4. 按 y 归并。
5. 只检查距离中线小于当前答案的点。
6. 对条带中的点按 y 枚举有限邻居。

复杂度：

$$O(n \log n)$$

少于两个点时不存在点对，模板返回正无穷。

常见问题与修正

1. 原文同时定义了参数完全相同、仅返回类型不同的两个 `Cross`，C++ 不允许仅按返回类型重载。整理稿拆分为 `cross` 和 `orientation`。
2. 原文若干函数连续写了两个 `return`，后一个永远不可达。整理稿用不同函数或枚举明确结果。
3. 原文 `Rotate` 的写法不直观，整理稿使用标准旋转矩阵。
4. 原文角度计算没有限制 `acos` 参数，可能因浮点误差得到 `NaN`。
5. 原文整点线段函数对水平线段和竖直线段直接返回 `1`，这是错误的。统一公式应为 `gcd(dx,dy)+1`。
6. 格点算法不应使用 `double Point`，否则 `abs`、`gcd` 和面积转换可能产生错误。
7. 原文最小圆覆盖没有处理空点集，且三点共线时会在外接圆计算中除以零。
8. 原文随机化使用多次 `rand()%n` 交换，整理稿改用 `std::shuffle`。
9. 原文自适应 Simpson 使用 `asr(1,r,0)`，会导致错误判定或无意义递归。
10. Simpson 的停止条件应与当前区间容差相关，并使用 `/15` 误差修正。
11. 原文圆与多边形面积交会原地平移输入多边形，造成调用后的数据被修改；整理稿不修改输入。
12. 浮点近似相等不适合直接用于 `std::sort` 比较器，否则可能破坏严格弱序。
13. 圆与圆重合时有无穷多个交点，不能用普通交点数组表达，应先判断位置关系。
14. 外接圆、内切圆、直线交点等接口均存在退化输入，整理稿通过 `bool` 或空数组报告失败。
15. 原文余弦定理第二、第三式的角标存在笔误，整理稿分别修正为 `cos B`、`cos C`。
16. 正多边形中心并不是任意多边形的重心；“外接圆圆心与重心重合”只适用于正多边形。

参考资料

- [计算几何模板整理 - 知乎](#)
- [计算几何的模板（大神整理） - CSDN](#)
- [判断点在多边形内的七种方法 - CSDN](#)
- [二维计算几何基础 - OI Wiki](#)
- [计算几何之求圆与直线的交点 - 知乎](#)
- [计算几何之圆与圆的交点 - 知乎](#)
- [求圆外一点到圆的切点坐标 - CSDN](#)
- [随机增量法 - OI Wiki](#)
- [计算三角形的外接圆 - 知乎](#)

- [570 向量运算、点线关系 - Bilibili](#)
- [571 叉积应用、线线关系 - Bilibili](#)
- [572 三角剖分、面积计算 - Bilibili](#)
- [579 自适应 Simpson 积分 - Bilibili](#)
- [577 平面最近点对、分治算法 - Bilibili](#)
- [平面最近点对分治算法 - 博客园](#)
- [圆的面积并（格林公式） - 博客园](#)